

Matrícula Tec – Sistema de Matrícula de Cursos Técnicos do CETAM

Programação Orientada a Objetos Java com Contextualização Amazônica

Estudantes: Aldenora Santos, Caio Felipe De Jesus, Joquebede Barboza, Sophia Roque Martins, Sarah Da Silva Siqueira, Suelen Tavares e Suely Nascimento Costa

Instrutora: Profª Priscila L. S. Gonçalves | CETAM | Manaus 2026

Contextualização Amazônica & Operação do Sistema

Contexto e Aplicação Prática no CETAM

Realidade Regional

O CETAM atende Manaus e diversos polos no interior do Amazonas com cursos técnicos organizados por módulos.

Desafio Operacional

A gestão manual de matrículas pode acarretar sobrecarga no limite de vagas das salas de aula e falta de controle do fluxo dos alunos.

Solução Automatizada

O MatriculaTec gerencia em memória a hierarquia de Curso -> Módulo -> Disciplina, controlando em tempo real o limite de vagas por Turma e o status da matrícula.

Arquitetura em Pacotes & Repositório GitHub

Divisão Estrutural do Projeto por Camadas

```
MATRICULATEC/  
└─ src/  
    └─ br/  
        └─ edu/  
            └─ cetam/  
                └─ matriculatec/  
                    ├── exception/  
                    │   ├── MatriculaDuplicadaException.java  
                    │   ├── RegistroNaoEncontradoException.java  
                    │   └── VagasEsgotadasException.java  
                    ├── main/  
                    │   └── Main.java  
                    ├── model/  
                    │   ├── enums/  
                    │   │   └── SituacaoMatricula.java  
                    │   ├── interfaces/  
                    │   │   └── RelatorioExportavel.java  
                    │   ├── Aluno.java  
                    │   ├── Curso.java  
                    │   ├── Disciplina.java  
                    │   ├── Matricula.java  
                    │   ├── Modulo.java  
                    │   ├── Pessoa.java  
                    │   └── Turma.java  
                    └─ service/  
                        └── MatriculaService.java
```

=====Resumo dos Pacotes (package) no topo de cada arquivo

Cada arquivo .java deve ter na sua primeira linha o caminho

Na pasta main:

```
package br.edu.cetam.matriculatec.main;
```

Na pasta exception:

```
package br.edu.cetam.matriculatec.exception;
```

Na pasta model:

```
package br.edu.cetam.matriculatec.model;
```

Na pasta enums:

```
package br.edu.cetam.matriculatec.model.enums;
```

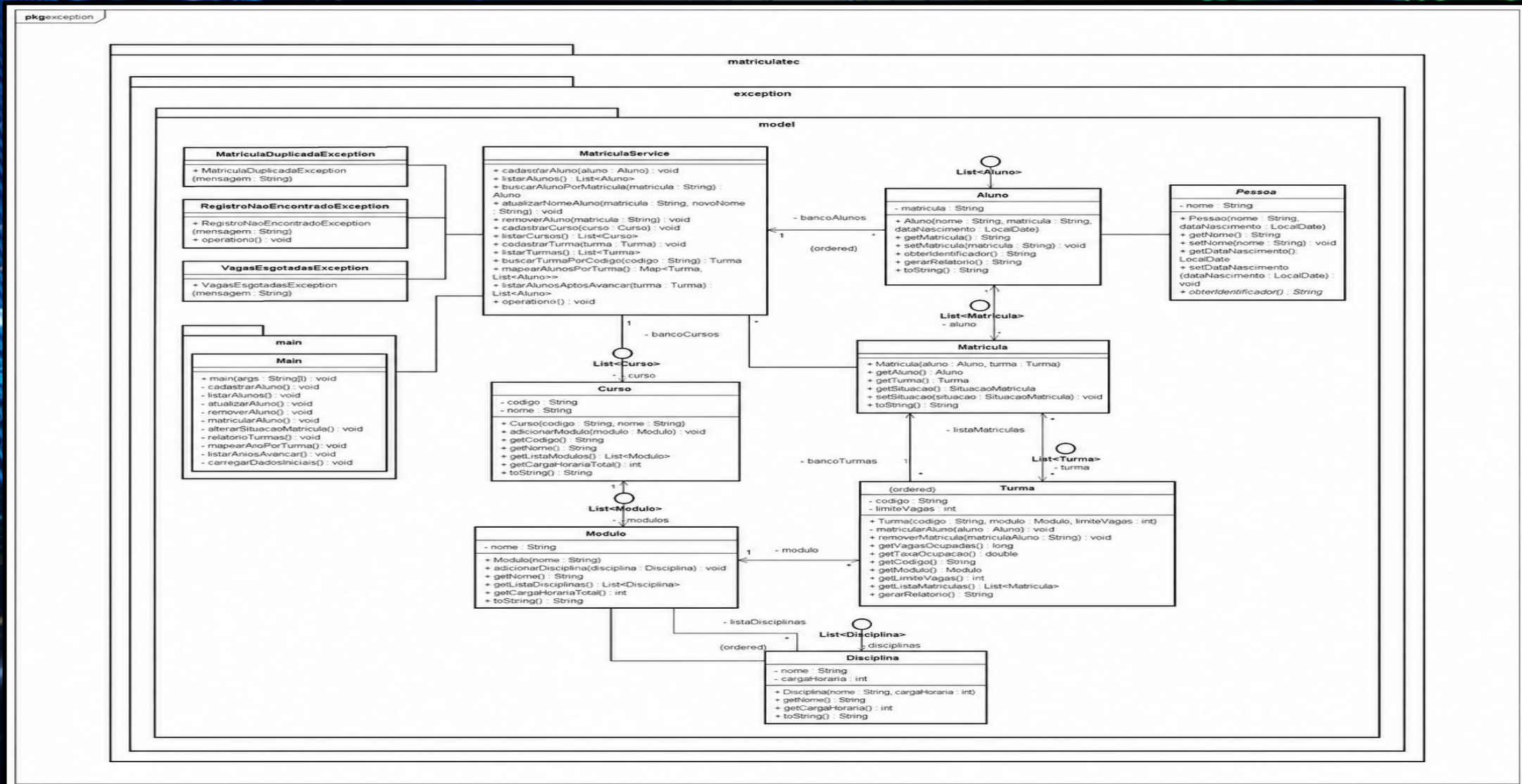
Na pasta interfaces:

```
package br.edu.cetam.matriculatec.model.interfaces;
```

Na pasta service:

```
package br.edu.cetam.matriculatec.service;
```

Modelagem UML – Diagrama de Classes



Mapeamento Completo das 14 Classes do Sistema

Nº	Pacote	Classe / Componente	Papel na Orientação a Objetos
1	model	Pessoa	Classe Abstrata (Base para dados pessoais)
2	model	Aluno	Classe Concreta (extends Pessoa e implements RelatorioExportavel)
3	model	Curso	Classe Concreta (Agrega lista de Modulo)
4	model	Modulo	Classe Concreta (Agrega lista de Disciplina)
5	model	Disciplina	Classe Concreta (Guarda nome e carga horária)
6	model	Turma	Classe Concreta (Gere vagas, matrículas e relatórios)
7	Matricula	Matricula	Classe Concreta (Associa Aluno, Turma e SituacaoMatricula)
8	interfaces	RelatorioExportavel	Interface (Declara o contrato gerarRelatorio())
9	enums	SituacaoMatricula	Enum (ATIVA, TRANCADA, CONCLUIDA, CANCELADA)
10	service	MatriculaService	Classe Service (Armazenamento em memória e buscas)
11	exception	VagasEsgotadasException	Exceção Checada (Regra de limite de vagas)
12	exception	MatriculaDuplicadaException	Exceção Checada (Regra de duplicidade)
13	exception	RegistroNaoEncontradoException	Exceção Checada (Regra de busca de registros)
14	main	Main	Classe Executável (Menu interativo no terminal)

Encapsulamento e Métodos de Acesso

Proteção de Atributos e Integridade

```
// Visualização VSCode - Turma.java
package br.edu.cetam.matriculatec.model;

import java.util.ArrayList;
import java.util.List;

public class Turma implements RelatorioExportavel {
    private String codigo;
    private Modulo modulo;
    private int limiteVagas;
    private List<Matricula> listaMatriculas;

    public Turma(String codigo, Modulo modulo, int limiteVagas) {
        this.codigo = codigo;
        this.modulo = modulo;
        this.limiteVagas = limiteVagas;
        this.listaMatriculas = new ArrayList<>();
    }

    public String getCodigo() { return codigo; }
    public int getLimiteVagas() { return limiteVagas; }
    public List<Matricula> getListaMatriculas() {
        return listaMatriculas;
    }
}
```

Atributos Fechados

Modificador private utilizado em 100% dos atributos das entidades do domínio.

Imutabilidade de Listas

Métodos como getListasModulos() utilizam Collections.unmodifiableList() para garantir a segurança dos dados.

Herança, Interface e Polimorfismo

Implementação em Tempo de Execução

```
// Visualização VSCode - Aluno.java
package br.edu.cetam.matriculatec.model;

import br.edu.cetam.matriculatec.model.interfaces.RelatorioExportavel;
import java.time.LocalDate;

public class Aluno extends Pessoa implements RelatorioExportavel {
    private String matricula;

    public Aluno(String nome, String matricula, LocalDate dataNascimento) {
        super(nome, dataNascimento);
        this.matricula = matricula;
    }

    @Override
    public String obterIdentificador() {
        return "MAT-ALUNO: " + matricula;
    }

    @Override
    public String gerarRelatorio() {
        return String.format(
            "Aluno: %-30s | Matrícula: %-10s | Data Nasc: %s",
            getNome(), matricula, getDataNascimento());
    }
}
```

A abstração e a herança

Pessoa estabelece a base com o método abstrato **obterIdentificador()**, implementado por Aluno.

Polimorfismo

Aluno e Turma implementam **RelatorioExportavel**, fornecendo saídas formatadas distintas para o método **gerarRelatorio()** em tempo de execução.

Coleções Java API (ArrayList e HashMap)

Estruturas de Armazenamento em Memória

```
// Visualização VSCode - MatriculaService.java
public Map<Turma, List<Aluno>> mapearAlunosPorTurma() {
    Map<Turma, List<Aluno>> mapa = new HashMap<>();
    for (Turma turma : bancoTurmas) {
        List<Aluno> alunos =
            turma.getListaMatriculas().stream()
                .filter(m -> m.getSituacao()
                    == SituacaoMatricula.ATIVA)
                .map(Matricula::getAluno)
                .collect(Collectors.toList());
        mapa.put(turma, alunos);
    }
    return mapa;
}
```

ArrayList

Utilizado no **MatriculaService** para gerenciar a coleção sequencial de alunos, turmas e cursos.

HashMap

Mapeamento em pares **Chave-Valor** (**Map<Turma, List>**) para listar de forma direta apenas os alunos com matrícula **ATIVA** por turma.

Tratamento de Exceções de Negócio

VagasEsgotadas Exception:

Disparada ao tentar matricular em turma com limite esgotado.

MatriculaDuplicada Exception:

Impede cadastros repetidos do aluno na mesma turma.

RegistroNaoEncontradoException:

Tratamento para buscas de registros inexistentes.

Tratamento no Console:

Blocos try-catch na classe Main evitam travamentos do sistema diante de entradas inválidas de dados pelo usuário.

Regra de Negócio Crítica & Exceção de Vagas

Validação do Limite de Vagas em Turma

```
// Visualização VSCode - Turma.java
public void matricularAluno(Aluno aluno)
    throws VagasEsgotadasException,
           MatriculaDuplicadaException {

    if (getVagasOcupadas() >= limiteVagas) {
        throw new VagasEsgotadasException(
            "Limite de vagas (" + limiteVagas +
            ") atingido na turma " + codigo);
    }

    boolean duplicado = listaMatriculas.stream()
        .anyMatch(m -> m.getAluno()
            .getMatricula()
            .equalsIgnoreCase(
                aluno.getMatricula()));
    if (duplicado) {
        throw new MatriculaDuplicadaException(
            "Aluno " + aluno.getNome() +
            " já está matriculado nesta turma.");
    }
    this.listaMatriculas.add(
        new Matricula(aluno, this));
}
```

Validação de Capacidade

A chamada **matricularAluno()** verifica **getVagasOcupadas() >= limiteVagas** antes de efetivar a inserção.

Disparo da Exceção

Lançamento da exceção checada customizada **VagasEsgotadasException** e checagem prévia de duplicidade com **MatriculaDuplicadaException**.

Terminal Console – CRUD (Atualizar, Remover e Captura de Erro)

Tratamento de Exceções e Manipulação de Dados em Tempo de Execução

Coluna 1 – Trecho de Captura no Código (Main.java):

```
// Visualização VSCode - Main.java
private static void matricularAluno() {
    try {
        Aluno aluno = service.buscarAlunoPorMatricula(mat);
        Turma turma = service.buscarTurmaPorCodigo(codTurma);
        turma.matricularAluno(aluno);
        System.out.println("[SUCESSO] Matrícula efetuada com sucesso!");
    } catch (VagasEsgotadasException e) {
        System.out.println("[ERRO NA MATRÍCULA] " + e.getMessage());
    }
}
```

Coluna 2 – Execução e Simulação de Erro no Terminal:

Escolha uma opção: 5

Matrícula do Aluno: 202607

Código da Turma: INF-M1

[ERRO NA MATRÍCULA] Limite de vagas (5) atingido na turma INF-M1

Escolha uma opção: 3

Digite a matrícula do aluno a atualizar: 202601

Digite o novo nome completo: CAIO FELIPE FREITAS DE JESUS

[SUCESSO] Nome atualizado com sucesso!

Terminal Console – Menu CRUD (Cadastrar e Listar)

Integração entre Código Main.java e Execução em Terminal

Trecho do Código (Main.java):

```
switch (opcao) {  
    case 1 -> cadastrarAluno();  
    case 2 -> listarAlunos();  
    case 5 -> matricularAluno();  
}  
  
private static void cadastrarAluno() {  
    System.out.print("Nome completo: ");  
    String nome = scanner.nextLine();  
    System.out.print("Matrícula: ");  
    String mat = scanner.nextLine();  
    service.cadastrarAluno(new Aluno(nome, mat,  
dataNasc));  
}
```

Execução no Terminal Console:

CETAM - MatriculaTec (Sistema de Matrículas)

1. Cadastrar Aluno | 2. Listar Alunos | 5.
Matricular

Escolha uma opção: 1

--- Cadastrar Novo Aluno ---

Nome completo: ALDENORA

Matrícula: 202607

Data de Nascimento (dd/mm/aaaa): 10/04/2004

[SUCESSO] Aluno cadastrado com sucesso!

Escolha uma opção: 2

--- Lista de Alunos ---

Aluno: ALDENORA | Matrícula: 202607

Lógica de Negócio – Taxa de Ocupação & Avanço de Módulo

Métodos de cálculo e filtros com Stream API

Coluna 1 – Código (Turma.java e MatriculaService.java)

```
// Visualização VSCode - Turma.java
public double getTaxaOcupacao() {
    if (limiteVagas == 0) return 0.0;
    return ((double) getVagasOcupadas() / limiteVagas) * 100;
}

// Visualização VSCode - MatriculaService.java
public List<Aluno> listarAlunosAptosAvancar(Turma turma) {
    return turma.getListaMatriculas().stream()
        .filter(m -> m.getSituacao() == SituacaoMatricula.CONCLUIDA)
        .map(Matricula::getAluno)
        .collect(Collectors.toList());
}
```

Coluna 2 – Fórmulas e Regras Aplicadas

Filtro de Avanço:

Utilização da Stream API para buscar alunos cujo atributo situacao seja igual a SituacaoMatricula.CONCLUIDA, permitindo identificar quem está apto a avançar para o próximo módulo.

Recursos e Ferramentas



GitHub

Repositório com código-fonte, documentação técnica e controle de versão do projeto

MatrículaTec Link:

[https://github.com/suelentavares/:](https://github.com/suelentavares/)



Trello

Gerenciamento de tarefas, sprints e acompanhamento do progresso do desenvolvimento. Link:

<https://trello.com/invite/b/6a56971be9f2d9a1d6be40bf/ATTI2cc1df2aeb8ceb0c19316ca84e9ab6648BDF0BB5/projeto-programacao-em-java>